

Vehicle Category Classification using VGG16 Machine Learning Model

Revathi A/P Thiyagarajan (A202248), Nancy Anne (A202459),
Kavi Priya A/P Balasupramaniam (A202660), Jeslyn A/P A Saminathan (A202747)
Set/Group: 2CS2/AInspire

Abstract — The demand for accurate and efficient vehicle classification models is growing with the rise of intelligent transportation systems. This project focuses on addressing this challenge using VGG16, a pre-trained convolutional neural network known for its robust feature extraction capabilities. The method is very well-known in image recognition field at the present because of its performance as well as its prior research that has extensively validated VGG16 for diverse image recognition tasks however its application to vehicle classification requires further exploration. Leveraging the Kaggle Vehicle Classification dataset, which comprises labelled images of various vehicle types, this study employs transfer learning to fine-tune VGG16 for optimal performance. The methodology includes rigorous preprocessing, data augmentation to enhance variability, and hyperparameter tuning to maximize accuracy. This project demonstrates the potential of VGG16 to contribute significantly to intelligent transportation systems by enabling reliable and scalable vehicle classification solutions. The result shows promising performance with the ability of the model to achieve high accuracy during training and validation phase which are 97% and 96% respectively. However, the validation accuracy and loss variations point to possible overfitting. The confusion matrix reveals reliable classification across most categories although there are some misclassifications highlighting areas for further improvement

Keywords — VGG16, vehicle categories, vehicle classification

I. INTRODUCTION

This project is centered around computer vision and deep learning in the realm of identifying and categorizing vehicles, such as cars, planes, ships, trains, and motorcycles for applications such as traffic control systems and autonomous vehicles, to improve road safety and traffic efficiency in cities. As environments expand and the global population continues to grow, there is an increasing need for effective vehicle identification systems that can operate with precision and automation. As reliance on human input errors decreases, the significance of automated visual data interpretation in the transportation and logistics sectors becomes increasingly important. This shift highlights how smart technologies are becoming more integrated into daily routines, enabling data-informed decisions. The project's information source comprises the "Vehicle Classification" dataset sourced from Kaggle that features images portraying vehicle varieties for analysis purposes, in machine learning applications effectively utilizing a learning model named VGG16, A Convolutional Neural Network (CNN). VGG16 is renowned for its image classification capabilities. Was employed after being fine-tuned with the dataset to enhance

accuracy while minimizing computational workload compared to starting model training, from scratch.

II. METHOD

A. Knowledge Acquisition

The dataset for this project is sourced from Kaggle, ensuring usability through well-structured data and metadata essential for analysis and modelling. It comprises 3,990 images of vehicles categorized into 5 classes: Cars (790 images), Motorcycles (800 images), Planes (800 images), Trains (800 images), and Ships (800 images), all of which are in JPEG format with the .jpg extension. These images capture vehicles in diverse settings, angles, and lighting conditions, reflecting real-world variability and complexity, which is essential for training robust classification models. However, the dataset poses challenges due to some images containing partial obstructions, overlapping vehicles, or low resolution. These characteristics highlight the importance of advanced preprocessing techniques to address potential obstacles in achieving high classification accuracy.

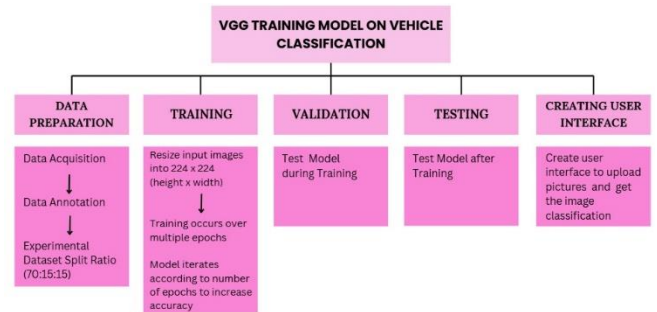


Figure 1 above shows VGG TRAINING MODEL ON VEHICLE CLASSIFICATION.

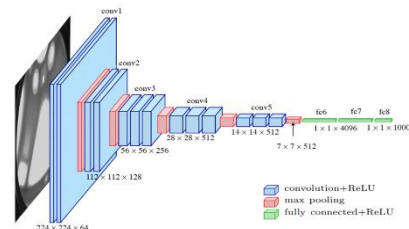


Figure 2 above shows ARHITECTURAL LAYER VGG16 TRAINING MODEL ON VEHICLE CLASSIFICATION.

B. Knowledge Representation

The knowledge representation for this vehicle classification project is organized utilizing a combination of Semantic Net, Frame, and Rule-based techniques to capture and organize the links between vehicle categories and their properties.

i) Knowledge Representation through Semantic Networks

A Semantic Net is selected for modelling the connections among vehicle types as it effectively represents hierarchical relationships and common characteristics. It depicts information in a graph-like format, where nodes signify concepts and edges illustrate connections.

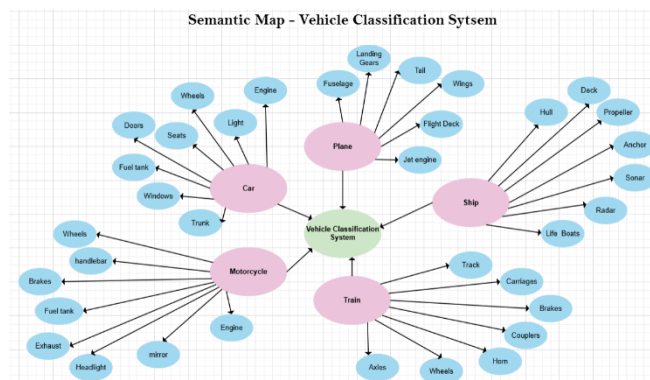


Figure 3 above shows SEMANTIC DIAGRAM.

Vehicles are the foundational notion, with branches leading to various vehicles such as cars, motorcycles, planes, ships and trains. Each vehicle type has its own set of distinguishing characteristics. For example, a car has wheels, doors, an engine, and windows. This structure illustrates the inheritance of common features like wheels in both cars and motorcycles and the unique features of each vehicle type such as fuselage for planes or track for trains.

Semantic Networks are used for inheritance and relationship analysis. Semantic nets successfully capture both inheritances, for example, cars and motorcycles both have the characteristic "wheels" and the distinct connections between vehicle types. This framework enables a simple understanding and identification of common and different properties across vehicle classifications. It makes it easier to add new vehicle types or features by keeping a clear hierarchical structure.

ii) Knowledge Representation using Frame Structure

The specific characteristics of every vehicle type are represented using a frame structure. Each vehicle type is represented as a frame with certain features and values that explain its characteristics. This is beneficial for organising data in an organised way, with each frame serving as a template for vehicle kinds.

Figure 4 below shows FRAME DIAGRAM

Vehicle Classification System
Superclass: General Vehicles
Attributes:
- Vehicle Type: {Car, Motorcycle, Train, Ship, Plane}
Relations:
- Contains subclasses for vehicle categories
Relations: Subtype of Vehicle Classification

Car
Superclass: Vehicle Classification System
Attributes:
- Wheels
- Engine
- Seats
- Windows
- Doors
- Fuel Tank
- Trunk
Relations: Subtype of Vehicle Classification

Motorcycle
Superclass: Vehicle Classification System
Attributes:
- Wheels
- Handlebar
- Fuel Tank
- Brakes
- Engine
- Headlight
- Exhaust
- Mirror
Relations: Subtype of Vehicle Classification

Plane
Superclass: Vehicle Classification System
Attributes:
- Tails
- Wings
- Fuselage
- Landing Gears
- Flight Deck
- Jet engine
Relations: Subtype of Vehicle Classification

Ship	Train
Superclass: Vehicle Classification System	Superclass: Vehicle Classification System
Attributes:	Attributes:
- Hull	- Wheels
- Deck	- Axles
- Propeller	- Carriages
- Anchor	- Tracks
- Sonar	- Couplers
- Radar	- Brakes
- Life Boats	- Horn
Relations: Subtype of Vehicle Classification	Relations: Subtype of Vehicle Classification

Each vehicle type such as a car or a plane represented as a frame with a set of attributes that describe its specific qualities. For example, the car frame includes qualities like as wheels, doors, and body shape, all of which are important when determining classification for the VGG16 models. The Frame structure is useful for representing the individual specifics of each vehicle type in a clear and controlled manner.

Frames are used because they are especially successful in organising the properties of each vehicle type, making future extensions of vehicle categories or traits easier. They also provide a modular approach to categorisation, which allows each frame to be readily adjusted or extended without impacting the whole system.

iii) Knowledge Representation using Rules

Rules establish logical requirements that specify how vehicles can be categorised based on certain visual characteristics. Rules enable the system to make judgements based on observations like the number of wheels and the existence of wings.

Rule-based Classification:

a. Car

IF vehicle has 4 wheels AND
IF vehicle has doors AND
IF vehicle has seats AND
IF vehicle has an engine AND
IF vehicle has windows AND
IF vehicle has a fuel tank AND
IF vehicle has a trunk AND
IF vehicle has lights
THEN classify as **CAR**

b. Motorcycle

IF vehicle has 2 wheels AND
IF vehicle has a handlebar AND
IF vehicle has an engine AND
IF vehicle has fuel tank AND
IF vehicle has an exhaust AND
IF vehicle has brakes AND
IF vehicle has headlights AND
IF vehicle has mirrors
THEN classify as **MOTORCYCLE**

c. Train

IF Vehicle runs on tracks AND
IF Vehicle has carriages AND
IF Vehicle has wheels AND
IF Vehicle has axles AND
IF Vehicle has brakes AND
IF Vehicle has a horn AND

IF Vehicle has couplers
THEN classify as **TRAIN**

d. Ship

IF vehicle has a hull AND
IF vehicle has a deck AND
IF vehicle has propellers AND
IF vehicle has an anchor AND
IF vehicle has radar AND
IF vehicle has sonar AND
IF vehicle has lifeboats
THEN classify as **SHIP**

e. Plane

IF vehicle has wings AND
IF vehicle has jet engines AND
IF vehicle has landing gear AND
IF vehicle has a fuselage AND
IF vehicle has tail AND
IF vehicle has a flight deck
THEN classify as **PLANE**

Rules provide the logical links between features and vehicle types. For example, if an image has four wheels and a boxy appearance, it is classed as a car. The IF-THEN rule structure allows for deductive reasoning, making it perfect for categorisation jobs that require specific criteria to be satisfied before reaching a conclusion.

Rules enhance decision-making based on observable features by giving unambiguous categorization criteria, making the process interpretable and trouble shootable. Furthermore, rules improve the transparency of the categorisation process by allowing users to understand why a specific vehicle type was assigned to a given image.

iv) Knowledge Representation using Resnet50

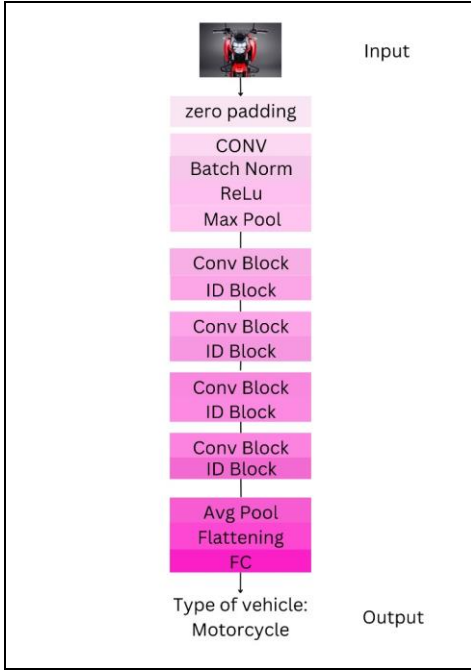


Figure 5 shows the Resnet50 model architecture

We used Resnet50 model to do our interface as it is also a part of CNN like VGG16 model. It eases the training of the deep networks. Firstly, it's the input layer where it accepts an image that's usually a size of 224 x 224(Height x Width). This layer standardizes the size of the input images for consistent processing. Then, zero padding layer adds padding to the input ensuring output of the initial convolution maintains the desired size. Then, convolution layer that extracts basic low-level features such as edges and textures. Next layer is batch normalization where it stabilizes and accelerate training. Next, rectified linear unit (ReLU) that reduces non-linearity and enables the model to learn complex features. Next, the max pooling layer further reduces the spatial dimensions. Each block after that reduces or restores the number of channels, extracts spatial features, expands or compresses the channel back respectively. Then, average pooling aggregates spatial information globally, reducing it to a single value. Then, flattening layer prepares the data for fully connected layer. Next, fully connected (FC) layer that maps the extracted features to the desired output classes which are Cars, Motorcycles, Plane, Ships and Trains. Lastly, the outputs the vehicles category and probabilities using SoftMax function.

SoftMax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

C. Machine Learning Techniques

The vehicle classification dataset is divided into 3 parts; training set, validation set, and test set with the ratio of (70:15:15). The training set should have a

high ratio (70) which is 560 for train, ship, motorcycle, planes and cars has a total of 553 data which results in 2793 training data as this set of data will be used for training the model to identify the different categories of vehicles. The training set comprises of a collection of inputs to ensure the model is trained in various environments and should anticipate any unobserved data sample that emerges in the future. Validation dataset was created by splitting the original dataset to evaluate the performance of the VGG16 model during training. Cars have a total of 118 data and the other 4 classes are 120 respectively and results in 598 total validation data. Additionally, for test data cars have a total of 119 and the rest have 120 images respectively which results in 599. The model's performance was monitored using validation accuracy and loss after each training epoch, which allowed for hyperparameter tuning and model optimization. This step ensured that the final model generalized well to unseen data. Performance was evaluated using metrics such as accuracy, precision, recall, and F1-Score, with overall accuracy reaching 97%. The VGG16 architecture consists of convolutional layers for feature extraction followed by fully connected layers for classification, optimized using backpropagation. The final layer was adjusted to classify 5 vehicle types. Despite achieving high accuracy, some misclassifications occurred due to overlapping or low-resolution images, highlighting areas for further optimization.

III. RESULT

Source Code

A. Interface DEMO

The VGG16 model was utilized for vehicle classification, predicting outcomes such as the type of vehicle present in an image. The classification results were accompanied by confidence scores generated using the SoftMax function, which assigns a probability to each class and represents the likelihood that the model has accurately classified the vehicle. These confidence scores, displayed as percentages, enhance user interpretability by quantifying the model's certainty in its predictions. The Graphical User Interface (GUI) for this project was built using Streamlit, a Python based library tailored for creating interactive and user-friendly web applications. Streamlit simplifies development with interactive widgets, seamless image uploads, and effortless integration with visualization libraries, allowing users to dynamically view classification results and explore the model's predictions. For model implementation and training, the project leveraged Torch (PyTorch), a versatile and open-source machine learning framework developed by Facebook AI Research. Torch supported efficient tensor computations, dynamic computational graph construction for flexible forward reasoning, and GPU acceleration, enabling faster and more efficient model training and inference. NumPy was employed for preprocessing and data manipulation tasks. By supporting N-dimensional array operations and mathematical computations, NumPy ensured that input images were resized, normalized, and properly scaled to match the requirements of the VGG16 model. This preprocessing step was critical for ensuring consistent input

quality and optimal model performance. Overall, the combination of Streamlit, Torch, and NumPy established a robust and efficient workflow for vehicle classification. It enabled accurate predictions with user-friendly interactivity, computationally optimized model execution, and seamless data preparation, making the system suitable for real-world vehicle recognition tasks in intelligent transportation and traffic monitoring systems.

DEMO VIDEO

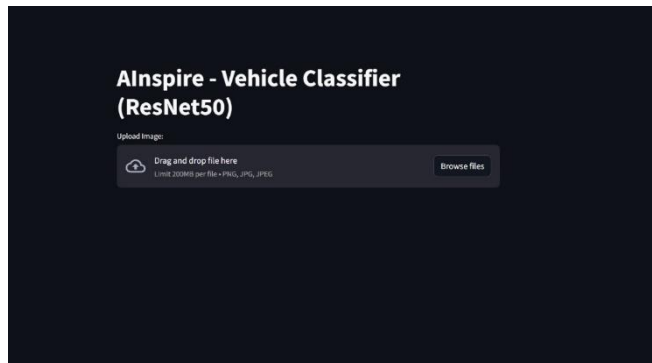


Figure 6 shows Interface Demonstration of Vehicle Classifier Using Restnet50

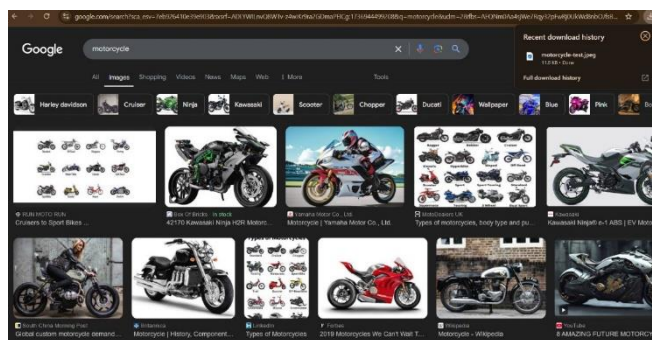


Figure 7 shows Pictures of Vehicles for Interface Demonstration

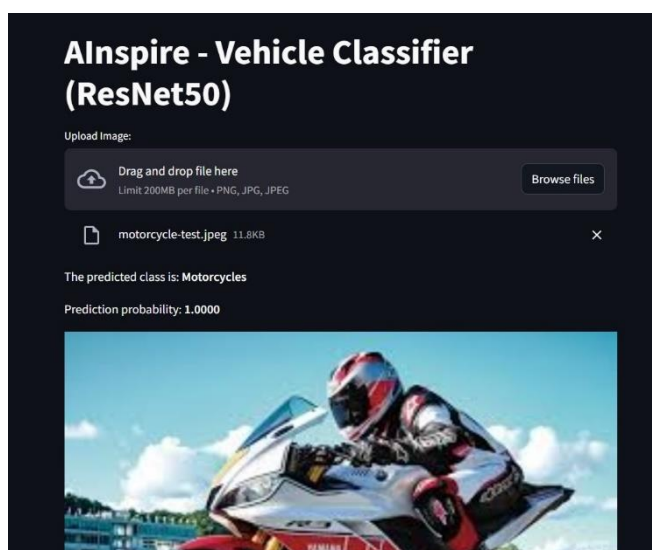


Figure 8 shows Prediction Probability for Motorcycle on Vehicle Classifier Using Restnet50

This artificial intelligence-powered car classifier analyses and classifies vehicle photos using a deep learning model known as ResNet50. A well-liked Convolutional Neural Network (CNN) architecture for image classification applications is called ResNet50 (Residual Neural Network with 50 layers). It works well for training deep networks because it makes use of "residual connections" to get around problems like vanishing gradients. To extract features like shapes, textures, and patterns, it processes incoming photos through several layers. When a user uploads an image to the tool, such as a picture of a car, the classification process begins. Preprocessing involves resizing and normalizing the image to meet the ResNet50 model's input specifications.

The image is then fed into the ResNet50 network, where early layers identify simple features like corners and edges, middle layers find more intricate patterns like windows, wheels, or vehicle contours, and later layers identify vehicle attributes that set cars, trucks, motorcycles, and other vehicles apart. Each specified class—such as "Motorcycles," "Cars," and "Trucks"—is given a probability by the network's last layer, and the class with the greatest probability is chosen as the predicted category. The model has 100 percent confidence in its prediction of "Motorcycles" in this instance.

To learn properties of automobiles, motorbikes, buses, and other vehicles, the ResNet50 model is fine-tuned on a dataset of vehicle photos after being pre-trained on a sizable data with millions of labelled images. The predicted class, for example, "Motorcycles" and the confidence score in this case, 1.0000, which indicates the model's level of assurance regarding the prediction, are produced by the tool. This technology's use applications include e-commerce classifying cars in listings or advertisements, insurance automatically classifying photographs of vehicle damage, traffic monitoring identifying vehicle kinds in real-time, and autonomous vehicles in identifying nearby vehicles for navigation. ResNet50 was selected because it can handle complicated picture data without overfitting or underfitting, and it strikes a compromise between accuracy and computational economy.

B. Machine Learning Result

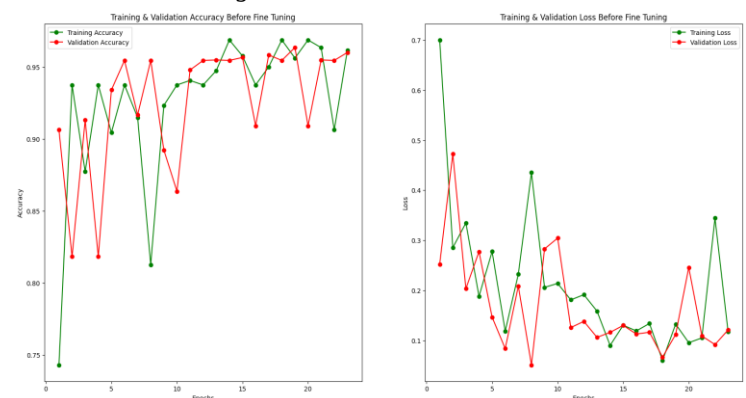


Figure 9 Training & Validation Accuracy and Training and Validation Loss

The graph training & validation accuracy (left) it compares training accuracy and validation accuracy across epochs before fine-tuning a model. It shows that both metrics improve overall, with occasional fluctuations, suggesting a well-performing model that avoids significant overfitting.

The training and validation loss (right) X-axis represents the number of epochs (training iterations) and Y- axis represents the accuracy values. Green line represents training loss while the red line represents validation loss

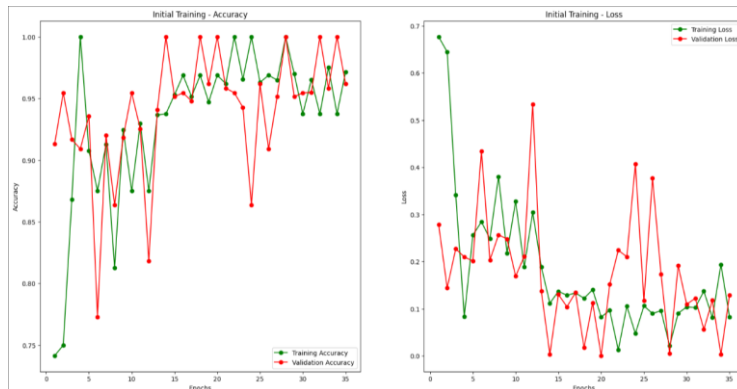


Figure 10 Initial Training – Accuracy (left) & Initial Training – Loss (right)

Left (Accuracy): Training and validation accuracy steadily improve, indicating effective learning. Some fluctuations in validation accuracy suggest minor variations in generalization.

Right (Loss): Training and validation loss decrease over epochs, showing model optimization. Occasional spikes in validation loss may indicate moments of overfitting.

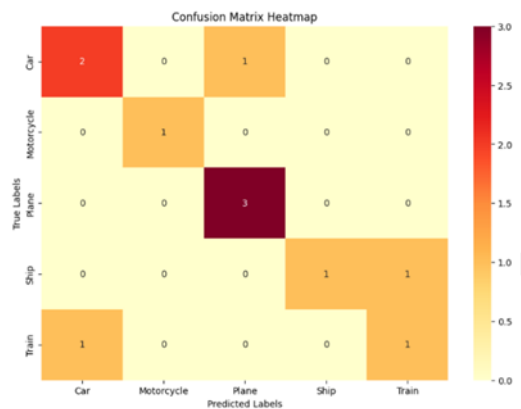


Figure 11:Confusion Matrix Heatmap

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{total samples}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

1. Car:

Precision	$= \frac{\text{TP}}{\text{TP} + \text{FP}}$ $= \frac{2}{2 + 1}$ $= \frac{2}{3}$ $= 0.67$
Recall	$= \frac{\text{TP}}{\text{TP} + \text{FN}}$ $= \frac{2}{2 + 1}$

	$= \frac{2}{3}$ ≈ 0.67
F1-score	$= \frac{2 * (\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$ $\approx \frac{2 * (0.67 * 0.67)}{(0.67 + 0.67)}$ ≈ 0.67
Accuracy	0.17
Specificity	0.89

The model still has room for improvement, particularly in reducing false positives or false negatives to boost the performance.

2. Motorcycle:

Precision	$= \frac{\text{TP}}{\text{TP} + \text{FP}}$ $= \frac{1}{1 + 0}$ $= \frac{1}{1}$ $= 1.00$
Recall	$= \frac{\text{TP}}{\text{TP} + \text{FN}}$ $= \frac{1}{1 + 0}$ $= \frac{1}{1}$ $= 1.00$
F1-score	$= \frac{2 * (\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$ $= \frac{2 * (1.00 * 1.00)}{(1.00 + 1.00)}$ $= 1.00$
Accuracy	0.20
Specificity	1.00

The model makes no errors in identifying "Motorcycle" category and has perfect classification for this class.

3. Plane:

Precision	$= \frac{\text{TP}}{\text{TP} + \text{FP}}$ $= \frac{3}{3 + 1}$ $= \frac{3}{4}$ $= 0.75$
Recall	$= \frac{\text{TP}}{\text{TP} + \text{FN}}$ $= \frac{3}{3 + 0}$ $= \frac{3}{3}$ $= 1.00$
F1-score	$= \frac{2 * (\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$ $= \frac{2 * (0.75 * 1.00)}{(0.75 + 1.00)}$ ≈ 0.86

Accuracy	0.20
Specificity	1.00

The model performs well in identifying positive instances but could still improve precision by reducing false positives.

4. Ship:

Precision	$= TP / (TP + FP)$ $= 1 / (1 + 0)$ $= 1 / 1$ $= 1.00$
Recall	$= TP / (TP + FN)$ $= 1 / (1 + 1)$ $= 1 / 2$ $= 0.50$
F1-score	$= 2 * (Precision * Recall) / (Precision + Recall)$ $= 2 * (1.00 * 0.50) / (1.00 + 0.50)$ ≈ 0.67
Accuracy	0.18
Specificity	0.91

The model still has room for improvement, particularly in reducing false positives or false negatives to boost the performance.

5. Train:

Precision	$= TP / (TP + FP)$ $= 1 / (1 + 1)$ $= 1 / 2$ $= 0.50$
Recall	$= TP / (TP + FN)$ $= 1 / (1 + 1)$ $= 1 / 2$ $= 0.50$
F1-score	$= 2 * (Precision * Recall) / (Precision + Recall)$ $= 2 * (0.50 * 0.50) / (0.50 + 0.50)$ $= 0.50$
Accuracy	0.17
Specificity	0.90

The model is performing at a moderate level but substantial room for improvement.

Significance of these metrics

Precision: Precision refers to how many of the expected positive cases are positive. A high precision suggests that the model generates few incorrect positive predictions.

Recall: Recall reflects how many positive examples the model properly identifies. A high recall indicates that the model is capturing most of the positive instances in the dataset.

F1-score: F1-score considers both false positives and false negatives, making it an appropriate statistic for unbalanced datasets with skewed accuracy and recall.

Loss Decrease Over Epochs

The loss often reduces with epochs, showing that the model is learning. However, there are variations, particularly in validation loss, indicating that the model's performance on previously unreported data varies.

Challenges: Underfitting or Overfitting

Overfitting: The validation accuracy and loss variations point to possible overfitting. The model works admirably on training data but less reliably on validation data.

Underfitting: Not visible since training accuracy is good and loss declines.

Final Metrics Summary

- Final Training Accuracy: Approximately 97%
- Final Validation Accuracy: Approximately 96%
- Final Loss: Training loss is approximately 0.1 and validation loss is also around 0.1.
- Total Epochs Run: 22

Model's Effectiveness and Challenges

The excellent training and validation accuracy reflect the model's ability to categorise vehicle photos. However, changes in validation accuracy suggest that further fine-tuning is required to achieve consistent performance across diverse datasets.

Before fine-tuning:

Classification	Accuracy (%)			
	Max	Min	Mean	SD
Type	0.9599	0.9247	0.94882	0.01249

After fine-tuning:

Classification	Accuracy (%)			
	Max	Min	Mean	SD
Type	0.9749	0.9648	0.9706	0.00377

IV. ANALYSIS

V. CONCLUSION

The VGG16 model, which classifies vehicles into five categories—cars, motorcycles, planes, ships, and trains—has proven useful in tackling real-world difficulties. The model used transfer learning to extract high-quality features from pre-trained convolutional layers, resulting in quick training and outstanding classification performance. Key preprocessing processes, such as scaling photos to 256x256 pixels, performing data augmentation, and converting them to tensor format, were required to address difficulties such as overlapping cars, partial obstructions, and uneven image resolutions. These methods guaranteed that the model could efficiently generalise across several circumstances while maintaining high accuracy across all five categories. The findings support VGG16's applicability for vehicle classification tasks with applications in intelligent transportation systems such as traffic flow optimisation, fleet management, and urban planning. Future studies may involve increasing the dataset, fine-tuning the model for certain subcategories, and optimising its deployment for real-time applications. This study emphasises the potential of deep learning to revolutionise transportation management and automation.

REFERENCES

- [1] L. Tang, "Image Classification Based On improved VGG Network," in *2021 IEEE 6th International Conference on Signal and Image Processing (ICSIP)*, 2021, pp. 316–320. doi: 10.1109/ICSIP52628.2021.9688778.
- [2] A. K. Mohamed, A. K. Ibrahim, A. Akram, A. Ibrahim, and M. Gamal, "Improving Vehicle Classification and Detection with Deep Neural Networks," in *2020 International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*, 2020, pp. 1–5. doi: 10.1109/ISAECT50560.2020.9523640.
- [3] M. Rahman, S. Rahman, and M. U. A. Ayoobkhan, "Fine-Tuned Convolutional Neural Networks for Bangladeshi Vehicle Classification," in *2022 International Conference on Innovations in Science, Engineering and Technology (ICISSET)*, 2022, pp. 421–426. doi: 10.1109/ICISSET54810.2022.9775889.
- [4] Y. Li and K. Deng, "Breast Cancer Identification Study Using Improved VGG," in *2023 8th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2023, pp. 467–470. doi: 10.1109/ICCCBDA56900.2023.10154755.
- [5] Z. Zheng and N. Slam, "A VGG Fire Image Classification Model with Attention Mechanism," in *2024 5th International Conference on Computer Vision, Image and Deep Learning (CVIDL)*, 2024, pp. 873–877. doi: 10.1109/CVIDL62147.2024.10603830.
- [6] F. Bajić, J. Job, and K. Nenadić, "Chart Classification Using Simplified VGG Model," in *2019 International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2019, pp. 229–233. doi: 10.1109/IWSSIP.2019.8787299.
- [7] T. Kaur and T. K. Gandhi, "Automated Brain Image Classification Based on VGG-16 and Transfer Learning," in *2019 International Conference on Information Technology (ICIT)*, 2019, pp. 94–98. doi: 10.1109/ICIT48102.2019.00023.
- [8] Mahardi, I.-H. Wang, K.-C. Lee, and S.-L. Chang, "Images Classification of Dogs and Cats using Fine-Tuned VGG Models," in *2020 IEEE Eurasia Conference on IOT, Communication and Engineering (ECICE)*, 2020, pp. 230–233. doi: 10.1109/ECICE50847.2020.9301918.
- [9] J. Shi and Y. Zhao, "Image classification optimization and results analysis based on VGG network," in *2020 IEEE International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA)*, 2020, pp. 1090–1099. doi: 10.1109/ICIBA50161.2020.9277329.
- [10] S. Mekruksavanich, P. Jantawong, and A. Jitpattanakul, "ResNet-based Deep Neural Network using Transfer Learning for Animal Activity Recognition," in *2022 6th International Conference on Information Technology (InCIT)*, 2022, pp. 445–449. doi: 10.1109/InCIT56086.2022.10067405.
- [11] Y. Qi, C. Baiyang, and L. Lan, "Deep Learning Based Image Recognition In Animal Husbandry," in *2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*, 2021, pp. 318–321. doi: 10.1109/ICCWAMTIP53232.2021.9674177.
- [12] C. Chai and Y. D. Wong, "Automatic vehicle classification and tracking method for vehicle movements at signalized intersections," in *2013 IEEE Intelligent Vehicles Symposium (IV)*, 2013, pp. 624–629. doi: 10.1109/IVS.2013.6629536.